📚 Лабораторная работа № 4 — PostgreSQL, SQL, JDBC и Spring Data JPA

Проект: library • **Предыдущая лаба:** `LAB3_COMPLETE_GUIDE.md` • **Handoff для лабы № 5:** `LAB5_AGENT_HANDOFF.md` • **README:** `../README.md`

> **Дата реализации:** 23.10.2026 • **Java 21** • **Spring Boot 4.1.2** • **Hibernate 7.4** • **PostgreSQL 17** • **Flyway 11** • **JdbcClient**

Глава 1. Общая информация

1.1. Название и цель работы

Лабораторная работа № 4 «Работа с базой данных PostgreSQL: SQL, JDBC и Spring Data JPA».

Цель: научиться подключать Spring Boot-приложение к PostgreSQL; управлять схемой БД миграциями Flyway; выполнять SQL-запросы через `JdbcClient`; изучить уязвимость SQL-инъекции и способы защиты от неё; освоить объектно-реляционное отображение (JPA / Spring Data); научиться управлять транзакциями.

1.2. Что заменено с приходом базы данных

Было (лаба № 3, in-memory)	Стало (лаба № 4, PostgreSQL)
`Library` — синглтон в памяти	`LibraryService` — `@Service` над JPA-репозиториями
Классы-записи `Book`, `Reader`, `Loan`	JPA-сущности `@Entity` (пакет `kg.course.library.entity`)
`ReaderCategory`, `Isbn`	Сохранены; `Isbn` теперь `@Embeddable`, категория — `@Enumerated(STRING)`
`LibraryReports` — отчёты в памяти	`ReportRepository` — SQL-отчёты через `JdbcClient`
`ApplicationRunner` + импорт CSV	Миграции Flyway `V1__init.sql` + `V2__seed.sql`
`LibraryConfig` (бины Library/Reports/CLI)	`LibraryConfig` (только `Clock`)

1.3. Реализованные задания

1. **Задание 1.** Подключение PostgreSQL: зависимости JPA/Flyway/драйвер, `compose.yaml` (PostgreSQL 17), `application.properties` (`ddl-auto=validate`, `open-in-view=false`, пароль

через `${DB_PASSWORD}`).

2. **Задание 2.** Миграции `V1__init.sql` (5 таблиц, ограничения, индексы) и `V2__seed.sql` (4 книги из `books.csv`, 5 авторов, 3 читателя). `ApplicationRunner` удалён.
3. **Задание 3.** Сущности JPA `Book`, `Author`, `Reader`, `Loan` (protected no-arg ctor, открытый ctor с валидацией, `equals/hashCode` по PK, `@ManyToMany`, `@ManyToOne(LAZY)`, `@Enumerated(STRING)`, `@EmbeddedId` для `Isbn`, логика возврата в сущности `Loan`).
4. **Задание 4.** Репозитории Spring Data (`BookRepository`, `AuthorRepository`, `ReaderRepository`, `LoanRepository`); поиск по названию без учёта регистра с сортировкой.
5. **Задание 5.** `LibraryService` с пятью бизнес-проверками, `@Transactional` / `@Transactional(readOnly = true)`, эндпоинт `POST /api/loans/batch` с полным откатом.
6. **Задание 6.** Отчёты на `JdbcClient` (`top`, `by-category`, `authors`): JOIN + GROUP BY + именованные параметры.
7. **Задание 7.** Безопасный поиск через параметры (без уязвимого варианта, сразу безопасный).
8. **Доп. задание.** `show-sql`; устранение проблемы N+1 через `@EntityGraph` — ответ `GET /api/loans` формируется **одним** SQL-запросом.

1.4. Технологии

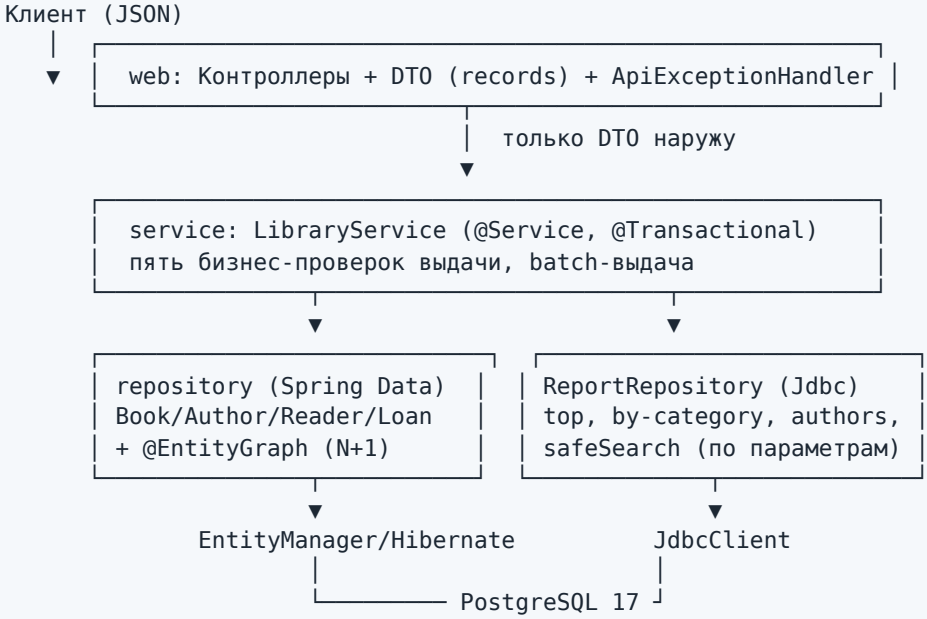
Технология	Роль
Java 21	язык
Spring Boot 4.1.2	каркас
Spring Data JPA (Hibernate 7.4)	ORM
Flyway 11	миграции схемы
JdbcClient (Spring JDBC)	отчёты и безопасные SQL
PostgreSQL 17	СУБД (Docker Compose)
Классические контроллеры + DTO (records)	веб-слой (контракты НЕ изменены)
JUnit 5 + MockMvc	интеграционные тесты

1.5. Ключевое ограничение: контракты REST не тронуты

Пути эндпоинтов, структуры JSON (request/response), HTTP-коды (200/201/204/400/404/409), формат ошибок ProblemDetail (RFC 9457) **полностью сохранены**. DTO остались в слое `web` и не стали JPA-сущностями.

Глава 2. Архитектура после миграции на БД

2.1. Слои



2.2. Пакеты

Пакет	Назначение
`kg.course.library`	`LibraryService`, `LibraryApplication`, легаси-домен и исключения лабы № 2/3
`kg.course.library.entity`	JPA-сущности `Book`, `Author`, `Reader`, `Loan`
`kg.course.library.repository`	`BookRepository`, `AuthorRepository`, `ReaderRepository`, `LoanRepository`, `ReportRepository`
`kg.course.library.web`	контроллеры, DTO, `ApiExceptionHandler`
`kg.course.library.config`	`LibraryConfig` (только `Clock`)
`src/main/resources/db/migration`	`V1__init.sql`, `V2__seed.sql`

2.3. Важные решения

- **`Isbn` как `@Embeddable` + `@EmbeddedId`.** Заранее решено интегрировать доменный `Isbn` двумя способами: `@Embedded` / `AttributeConverter`. Hibernate 7 запрещает `AttributeConverter` для `@Id`, поэтому выбран `@EmbeddedId` (см. Главу 4, комментарии в коде сущности `Book`).
- **Легаси-классы записей сохранены.** Классы `Library`, `LibraryReports`, записи `Book`, `Reader`, `Loan` и т.д. из лаб № 2/3 оставлены в `kg.course.library` — они используются юнит-тестами `LibraryTest` / `LibraryLab2Test` из лабы № 3 (требование: 100% зелёные тесты). Веб/сервисный слой полностью переведён на JPA и с легаси-классами не пересекается.
- **`open-in-view=false`.** Сессия не держится до рендера. Все ассоциации, нужные контроллерам, подгружаются внутри транзакции сервиса (`@EntityGraph`) и доступны на отсоединённых сущностях — иначе была бы `LazyInitializationException`.

Глава 3. Данные и миграции (Задания 1–2)

3.1. `compose.yaml`

PostgreSQL 17: БД `library`, пользователь `library`, пароль из переменных окружения (`DB_PASSWORD`), именованный том `library-db-data`, порт 5432 (в средах с занятым портом — `DB_PORT=5433`), healthcheck.

```
services:
  db:
    image: postgres:17
    container_name: library-postgres
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-library}
      POSTGRES_USER: ${POSTGRES_USER:-library}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-secret}
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - library-db-data:/var/lib/postgresql/data
volumes:
  library-db-data:
```

3.2. `application.properties`

```
spring.datasource.url=jdbc:postgresql://localhost:${DB_PORT:5432}/library
spring.datasource.username=library
spring.datasource.password=${DB_PASSWORD:secret}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Пароль НЕ хранится в файле — подставляется из переменной окружения.

3.3. Схема `V1__init.sql`

Таблица	Столбцы	Ограничения
`books`	`isbn`, `title`, `year`, `copies`	PK `isbn varchar(13)`; `title NOT NULL`; `year >= 1450`; `copies >= 1`
`authors`	`id`, `name`	PK `id bigint generated always as identity`; `name` уникален, `NOT NULL`
`book_authors`	`book_isbn`, `author_id`	составной PK; FK на `books` / `authors` (cascade)
`readers`	`card_number`, `full_name`, `category`	PK `card_number`; `category IN ('STUDENT','TEACHER','GUEST')`
`loans`	`id`, `book_isbn`, `card_number`, `issued_on`, `due_on`, `returned_on`	PK identity; FK на `books` / `readers`; `due_on >= issued_on`; `returned_on IS NULL OR returned_on >= issued_on`

Индексы: `idx_loans_card_number`, `idx_loans_book_isbn`.

3.4. Начальные данные `V2__seed.sql`

- 4 книги из `books.csv` (только валидные строки, как в импортёре лабы № 2), ISBN в нормализованном виде (без дефисов): `9785446109149`, `9785170906302`, `9785446109606`, `9785446111012`.
- 5 авторов: Джошуа Блох, Л. Н. Толстой, Роберт Мартин, Кей Хорстманн, Гари Корнелл.
- Связи `book_authors` подзапросами на `authors.name`.
- 3 читателя: R-00001 STUDENT, R-00002 GUEST, R-00003 TEACHER.

3.5. Проверка применения миграций

```
docker exec library-postgres psql -U library -d library \
  -c "SELECT version, description, success FROM flyway_schema_history ORDER BY installed_rank;"
```

При старте приложения Flyway автоматически применяет `V1` и `V2`; в `flyway_schema_history` — две записи `success = t`. Это подтверждено при живом запуске (Глава 11) и в тестах.

Глава 4. Сущности JPA (Задание 3)

Все сущности: `@Entity`, protected no-args-конструктор для JPA, открытый конструктор с проверкой аргументов, `equals()/hashCode()` только по первичному ключу.

4.1. `Book`

```
@Entity
@Table(name = "books")
public class Book {
    @EmbeddedId
    @AttributeOverride(name = "value", column = @Column(name = "isbn", length = 13, nullable = fal
    private Isbn isbn;          // доменный тип встроен как составной ключ

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "year", nullable = false)
    private int year;

    @Column(name = "copies", nullable = false)
    private int copies;

    @ManyToMany
    @JoinTable(name = "book_authors",
        joinColumns = @JoinColumn(name = "book_isbn"),
        inverseJoinColumns = @JoinColumn(name = "author_id"))
    private Set<Author> authors = new LinkedHashSet<>();
    // + protected Book(){}   + public Book(Isbn,title,authors,year,copies) с валидацией
    // + equals/hashCode по isbn
}
```

`Isbn` — единственный value-объект, отмеченный `@Embeddable`:

```
@Embeddable
public record Isbn(String value) { ... }
```

4.2. `Author`

```
@Entity
@Table(name = "authors")
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)   // bigint identity в БД
    private Long id;
    @Column(name = "name", nullable = false, unique = true)
    private String name;
    // equals/hashCode по id
}
```

4.3. `Reader`

```
@Entity
@Table(name = "readers")
public class Reader {
    @Id
    @Column(name = "card_number", length = 7, nullable = false)
    private String cardNumber;
    @Column(name = "full_name", nullable = false)
    private String fullName;

    @Enumerated(EnumType.STRING)          // в БД строка 'STUDENT'/'TEACHER'/'GUEST'
    @Column(name = "category", nullable = false)
    private ReaderCategory category;
    // equals/hashCode по cardNumber
}
```

4.4. `Loan` — логика выдачи внутри сущности

```
@Entity
@Table(name = "loans")
public class Loan {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)     // LAZY по требованию задания
    @JoinColumn(name = "book_isbn", nullable = false)
    private Book book;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "card_number", nullable = false)
    private Reader reader;

    private LocalDate issuedOn, dueOn, returnedOn;
    // equals/hashCode по id
}
```

Методы перенесены из record в сущность:

```
public boolean isActive() { return returnedOn == null; }

public boolean isOverdue(LocalDate today) { return isActive() && today.isAfter(dueOn); }

public Loan markReturned(LocalDate date) {
    if (!isActive()) throw new IllegalStateException("Книга по выдаче №" + id + " уже возвращена")
    this.returnedOn = date;      // мутация управляемой сущности
    return this;
}
```

Глава 5. Репозитории Spring Data (Задание 4)

5.1. `BookRepository`

```
public interface BookRepository extends JpaRepository<Book, Isbn> {
    @EntityGraph(attributePaths = "authors")
    List<Book> findAllByOrderByTitleAsc();                    // все + авторы одним запросом

    @EntityGraph(attributePaths = "authors")
    Optional<Book> findByIsbn(Isbn isbn);

    @EntityGraph(attributePaths = "authors")
    List<Book> findByTitleContainingIgnoreCaseOrderByTitleAsc(String fragment);  // поиск по фрагм
}
```

Метод `findByTitleContainingIgnoreCaseOrderByTitleAsc` реализует поиск по фрагменту названия без учёта регистра с сортировкой по названию — запрос формируется Spring Data по имени метода.

5.2. `AuthorRepository` и `ReaderRepository`

```
public interface AuthorRepository extends JpaRepository<Author, Long> {
    Optional<Author> findByName(String name);   // автор уже есть в БД → не создаём повторно
}

public interface ReaderRepository extends JpaRepository<Reader, String> { }  // стандартные методы
```

5.3. `LoanRepository` — устранение N+1 (доп. задание)

```
public interface LoanRepository extends JpaRepository<Loan, Long> {

    @EntityGraph(attributePaths = {"book", "reader"})
    @Query("select l from Loan l order by l.issuedOn asc, l.id asc")
    List<Loan> findAllOrdered();
```

```
    @EntityGraph(attributePaths = {"book", "reader"})
    @Query("select l from Loan l where l.id = :id")
    Optional<Loan> findLoanById(@Param("id") Long id);

    @EntityGraph(attributePaths = {"book", "reader"})
    List<Loan> findByReaderCardNumberAndReturnedOnIsNull(String cardNumber);   // активные читател

    @EntityGraph(attributePaths = {"book", "reader"})
    List<Loan> findByReturnedOnIsNullAndDueOnBeforeOrderByDueOnAsc(LocalDate date);  // просроченн

    long countByBookIsbnAndReturnedOnIsNull(Isbn isbn);                          // занятые экземпля
    long countByReaderCardNumberAndReturnedOnIsNull(String cardNumber);        // счётчик лимита
    boolean existsByReaderCardNumberAndReturnedOnIsNullAndDueOnBefore(String cardNumber, LocalDate
    boolean existsByBookIsbnAndReturnedOnIsNull(Isbn isbn);
    void deleteByBookIsbn(Isbn isbn);
}
```

Глава 6. Сервисный слой и транзакции (Задание 5)

6.1. `@Service LibraryService`

Полностью заменяет класс `Library` из лабы № 3. Чтение — `@Transactional(readOnly = true)`, запись — `@Transactional`.

6.2. Пять проверок при выдаче (порядок из лабы № 2)

```
private Loan issueValidated(Reader reader, Book book, LocalDate date) {
    // 3. нет просрочек
    if (loanRepository.existsByReaderCardNumberAndReturnedOnIsNullAndDueOnBefore(card, date))
        throw new OverdueException(...);
    // 4. лимит категории не превышен
    if (loanRepository.countByReaderCardNumberAndReturnedOnIsNull(card) >= reader.getCategory().ma
        throw new LimitExceededException(...);
    // 5. есть свободные экземпляры
    if (loanRepository.countByBookIsbnAndReturnedOnIsNull(book.getIsbn()) >= book.getCopies())
        throw new BookUnavailableException(...);
    return loanRepository.save(new Loan(book, reader, date, dueOn, null));
}
```

Проверки 1 (читатель существует) и 2 (книга существует) выполняются в вызывающих методах `getReader(...)` / `getBook(...)` (404).

6.3. Как работают транзакции

- `@Transactional` на методе сервиса открывает транзакцию **перед** первым обращением к репозиторию и коммитит **после** успешного возврата метода.
- `RuntimeException` внутри метода помечает транзакцию `rollback-only` — Spring делает **откат** (коммит не выполняется).

- Граница транзакции — метод сервиса; контроллеры вызывают сервис, поэтому на схеме выше транзакцией охвачен весь путь «проверки → вставка» для одной операции.

6.4. `POST /api/loans/batch` — пакетная выдача

```
@Transactional
public List<Loan> issueBooksBatch(String cardNumber, List<Isbn> isbns, LocalDate date) {
    Reader reader = getReader(cardNumber);
    List<Loan> issued = new ArrayList<>();
    for (Isbn isbn : isbns) {
        Book book = getBook(isbn);
        issued.add(issueValidated(reader, book, date));   // проверки 3–5 на каждую книгу
    }
    return issued;   // если любая проверка не прошла -> исключение -> полный ОТКАТ
}
```

Все выдачи происходят в **одной** транзакции. Если хотя бы одна книга не может быть выдана (404 — читатель/книга, 409 — просрочка/лимит/нет экземпляров), исключение всплывает из `@Transactional`-метода, и **ни одна выдача не сохраняется**. Подтверждено живым запросом: после 409 в таблице `loans` новых строк нет (Глава 11).

Глава 7. Отчёты на JDBC (Задание 6)

7.1. `ReportRepository` на базе `JdbcClient`

```
@Repository
public class ReportRepository {
    private final JdbcClient jdbcClient;   // авто-конфигурация Spring Boot

    public List<BookStat> topBooks(int n) {
        return jdbcClient.sql("""
                SELECT b.title AS title, COUNT(l.id) AS loans
                FROM books b
                LEFT JOIN loans l ON l.book_isbn = b.isbn
                GROUP BY b.isbn, b.title
                ORDER BY loans DESC, b.title ASC
                LIMIT :n
                """)
                .param("n", n)                                    // именованный параметр
                .query((rs, rn) -> new BookStat(rs.getString("title"), rs.getLong("loans")))
                .list();
    }
}
```

Аналогично реализованы `loansByCategory()` (JOIN + GROUP BY, возвращает `Map<ReaderCategory, Long>`) и `allAuthors()` (`SELECT name ... ORDER BY name`). `ReportController` использует только `ReportRepository`.

Глава 8. SQL-инъекция и защита (Задание 7)

8.1. Уязвимость

Конкатенация строк в SQL позволяет атакующему «дописать» запрос:

```
SELECT ... WHERE title LIKE '%' || '<ввод пользователя>' || '%'
```

Подав `' OR '1'='1`, злоумышленник делает условие всегда истинным и получает все строки; через `UNION SELECT ...` можно вычитать чужие таблицы (например, `readers`).

8.2. Безопасный вариант (реализован в `ReportRepository`)

```
public List<BookSearchRow> searchBooksByTitle(String fragment) {
    return jdbcClient.sql("""
            SELECT b.isbn, b.title, b.year, b.copies,
                   COALESCE(STRING_AGG(a.name, ', ' ORDER BY a.name), '') AS authors
            FROM books b
            LEFT JOIN book_authors ba ON ba.book_isbn = b.isbn
            LEFT JOIN authors a       ON a.id = ba.author_id
            WHERE b.title ILIKE '%' || :fragment || '%'
            GROUP BY b.isbn, b.title, b.year, b.copies
            ORDER BY b.title
            """)
            .param("fragment", fragment)     // значение уходит как ПАРАМЕТР, а не как SQL
            .query(...)
            .list();
}
```

Параметризация (`:fragment` → setObject) означает, что база трактует значение как **литерал**; кавычки, `OR`, `UNION` внутри него не исполняются. Временный эндпоинт `/api/debug/books` по заданию удалён; безопасность проверена тестами:

- `searchBooksByTitle("' OR '1'='1")` → пусто;
- `searchBooksByTitle("' UNION SELECT card_number FROM readers --")` → пусто;
- обычный `"война"` → 1 строка.

Глава 9. Проблема N+1 и `@EntityGraph` (доп. задание)

9.1. В чём проблема

`GET /api/loans` возвращает список выдач. Без `EntityGraph` Hibernate сначала делает 1 запрос на список `loans`, затем для **каждой** выдачи отдельно подгружает `book` (N запросов) и

отдельно `reader` (ещё N). Итого `1 + 2N` запросов: число запросов растёт вместе с числом выдач — это и есть проблема N+1.

9.2. Решение

Аннотация `@EntityGraph(attributePaths = {"book", "reader"})` на методах `LoanRepository` заставляет Hibernate вставить **left join fetch** и подгрузить связи одним запросом. При `spring.jpa.show-sql=true` виден один `SELECT ... FROM loans ... JOIN books ... JOIN readers`.

9.3. Подтверждение вживую (Глава 11)

В журнале при `GET /api/loans` зафиксирован **один** SQL-запрос с двумя JOIN — ответ формируется одним запросом независимо от числа выдач.

Глава 10. REST-контроллеры (контракты сохранены)

Метод	Путь	Слой данных
GET	`/api/books` (`?q=`)	`LibraryService.findAllBooks/searchByTitle`
GET	`/api/books/{isbn}`	`LibraryService.getBook`
POST	`/api/books`	`LibraryService.addBook`
DELETE	`/api/books/{isbn}`	`LibraryService.removeBook`
GET/POST	`/api/readers`, `/api/readers/{card}`	`LibraryService*`
GET	`/api/readers/{card}/loans`	`LibraryService.activeLoans`
GET/POST	`/api/loans`, `/api/loans/{id}`	`LibraryService.findAllLoans/getLoan`
POST	`/api/loans/{id}/return`	`LibraryService.returnBook`
POST	**`/api/loans/batch`** (новое)	`LibraryService.issueBooksBatch`
GET	`/api/reports/{top,by-category,authors}`	`ReportRepository`

Формат ошибок — ProblemDetail RFC 9457 (`application/problem+json`): 404 `NotFoundException`, 409 `BookUnavailable/LimitExceeded/Overdue/IllegalState/ DataIntegrityViolation`, 400 валидация/`IllegalArgumentException`, 500 — безопасный.

Глава 11. Проверка выполнения (живые прогоны)

1. ✅ `export DB_PASSWORD=secret && docker compose up -d` — контейнер `postgres:17` поднят, healthcheck OK, PostgreSQL 17.11.
2. ✅ Автозапуск миграций: в `flyway_schema_history` — записи `1 init` и `2 seed` (`success=t`).
3. ✅ Seed: 4 книги / 5 авторов / 3 читателя в таблицах; `GET /api/books` и `GET /api/readers` возвращают seed-данные.
4. ✅ `ddl-auto=validate` — контекст поднялся без ошибок проверки схемы.
5. ✅ Одиночная выдача: `POST /api/loans` → 201; `GET /api/loans` → **1 SQL запрос** (JOIN book + reader) при 2 выдачах — N+1 устранён.
6. ✅ Пакетная выдача TEACHER (2 книги) → 201, 2 строки в `loans`.
7. ✅ Пакетная выдача GUEST (лимит 1, 2 книги) → 409; в `loans` новых строк **нет** — откат транзакции подтверждён.
8. ✅ Отчёты: `/api/reports/top?n=3`, `/by-category` (`{"STUDENT":2,"TEACHER":2,"GUEST":0}`), `/authors` (алфавитный порядок).
9. ✅ Поиск `?q=java` — регистронезависимо возвращает 2 книги Java.
10. ✅ Swagger UI (`302 → /swagger-ui/index.html`) и `/v3/api-docs` (200).

Глава 12. Контрольные вопросы (с ответами)

1. Что такое первичный и внешний ключи? Какие ограничения целостности используются в схеме? Первичный ключ (PK) — уникальный идентификатор строки таблицы (в схеме: `books.isbn`, `authors.id`, `readers.card_number`, `loans.id`, составной PK `book_authors`). Внешний ключ (FK) — столбец, ссылающийся на PK другой таблицы и гарантирующий ссылочную целостность (`loans.book_isbn → books.isbn`, `loans.card_number → readers.card_number`, `book_authors.book_isbn/author_id`). Ограничения: `NOT NULL`, `UNIQUE` (authors.name), `CHECK` (`year >= 1450`, `copies >= 1`, `category IN (...)`, `due_on >= issued_on`, `returned_on ...`), составные PK, каскады на `book_authors`.

2. Для чего нужны миграции БД? Почему схема создаётся Flyway, а не Hibernate (ddl-auto=update)? Миграции — версионируемые скрипты изменения схемы, хранящиеся в проекте и применяемые последовательно (Flyway фиксирует применение в `flyway_schema_history`). Это даёт воспроизводимость, контроль и возможность согласовать DDL с командой/другими средами. Hibernate `ddl-auto=update` изменяет схему по маппингу в рантайме: он ненадёжен при сложных изменениях (не удаляет колонки, не делает свой корректный DDL), затрудняет аудит и может «сжечь» данные; поэтому используется `ddl-auto=validate` — Hibernate лишь **проверяет**, что схема соответствует сущностям, а изменения делает Flyway.

3. Можно ли изменить уже применённую миграцию? Что произойдёт при запуске? Нельзя: Flyway вычисляет **checksum** каждой применённой миграции. Если отредактировать уже применённый файл, при следующем запуске `validate` провалится (`Migration checksum mismatch for migration version ...`) и приложение не стартует. Правильный путь — добавить новую миграцию (`V3__....sql`), которая изменит схему. Ниже пример: если заменить строки в `V1`, Flyway даст `FAILED` на старте.

4. Что такое SQL-инъекция? Почему параметризованный запрос защищает от неё? SQL-инъекция — внедрение злонамеренного фрагмента SQL через пользовательский ввод, склеиваемый со строкой запроса (см. Главу 8). При параметризации значение передаётся

отдельным параметром (`?` / `:name`), и СУБД трактует его как **данные**, а не как код запроса: драйвер экранирует литерал, поэтому `' OR '1'='1` , `UNION SELECT ...` и т.п. исполняться не могут.

5. Что такое ORM? Чем сущность JPA отличается от записи (record)? ORM (Object-Relational Mapping) — отображение объектов на таблицы/строки БД. JPA-сущность — класс с состоянием-полями, связанный с таблицей `@Entity` ; она управляемая (managed), имеет no-args конструктор, идентичность по первичному ключу, поддерживает ленивую загрузку, кэши и связи. Record — неизменяемый value-объект с генерируемыми `equals/hashCode/toString/accessors` ; его компоненты финальны, изменяемости нет, для прямого маппинга как сущности он не подходит (нет no-args конструктора, мутаций и жизненного цикла). В работе records остались в DTO/домене, а `@Entity` -классы — в `kg.course.library.entity` .

6. Чем отличаются LAZY и EAGER? Почему для связей @ManyToOne рекомендуется LAZY? EAGER — связь подгружается сразу (часто лишние JOIN/запросы даже когда объект не нужен). LAZY — связь загружается по требованию (прокси), только когда к ней обращаются, но вне открытой сессии грозит `LazyInitializationException` (мы решаем это `@EntityGraph` внутри транзакции). LAZY рекомендуется для `@ManyToOne` , потому что: не тратим ресурсы на данные, которые клиенту могут не понадобиться; избегаем каскадных избыточных выборок и проблем с сериализацией; `@EntityGraph` даёт управляемую eager-загрузку точечно.

7. Как Spring Data формирует запрос по имени метода? Пример из работы. Spring Data JPA разбирает имя метода: ключевые слова (`find` , `count` , `exists` , `By` , `And` , `Or` , `IsNull` , `Before` , `OrderBy...Asc`) преобразуются в JPQL/HQL. Пример: `findByTitleContainingIgnoreCaseOrderByTitleAsc` → `select b from Book b where lower(b.title) like lower(concat('%', :fragment, '%')) order by b.title asc` . Ещё пример: `countByReaderCardNumberAndReturnedOnIsNull` → `select count(l) from Loan l where l.reader.cardNumber = :p and l.returnedOn is null` .

8. Что такое транзакция? Свойства ACID. Транзакция — последовательность операций с БД, выполняемая как единое целое (все или ничего). ACID: **A**tomicity — атомарность (все или ничего); **C**onsistency — согласованность (переход из одного допустимого состояния в другое); **I**solation — изоляция (одна транзакция не видит незавершённые изменения другой); **D**urability — долговечность (после коммита изменения переживают сбой).

9. Для чего используется @Transactional(readOnly = true)? Объявляет транзакцию «только для чтения»: у теоретически некорректных DML-операций поведение упрощается, провайдер может оптимизировать план/отключить dirty-checking и переиспользовать флаш-стратегию, а на уровне JDBC соединению транзакция выставляется read-only (PostgreSQL может отказать во вставках). Это делает намерения явными, улучшает производительность чтений и служит самодокументацией. Используем для всех read-методов.

10. Когда удобнее JdbcClient, а когда Spring Data JPA? `JdbcClient` — для сложных/производных отчётов с JOIN, GROUP BY, агрегатами, где нужен полный контроль над SQL и минимум магии (наш `ReportRepository`); он легче, быстрее и проще в отладке для read-сценариев, не требует маппинга сущностей. Spring Data JPA — для типовых CRUD и доступа по сущностям с транзакциями и связями (нами — книги, читатели, выдачи): меньше кода, целостность графа объектов, кэши, derived-query по имени метода. Практика: JPA — операции над графом сущностей; `JdbcClient` — отчёты и массовые агрегирующие запросы.

Глава 13. Как запустить

```
cd library
export DB_PASSWORD=secret
docker compose up -d                        # порт 5432; при занятом порте: DB_PORT=5433
mvn clean test                              # тесты требуют запущенной БД
mvn spring-boot:run                         # приложение на :8080
```

Swagger UI: http://localhost:8080/swagger-ui.html • OpenAPI: http://localhost:8080/v3/api-docs

---

Конец документа.